

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

ЛАБОРАТОРНА РОБОТА №4
“ «Основні типи змінних. Операції та їх пріоритет».”

Виконав:

Студент ФЕП-13
Беднарський Максим

Перевірив:

асис. Васюта В. М.

Львів 2026

Мета роботи: Вивчити поняття: основні типи змінних, вказівники, операції та їх пріоритет

Хід роботи

Завдання 1 Арифметичні, логічні та побітові оператори

На скріншоті наведено код мовою Сі, який демонструє роботу трьох груп операторів: арифметичних (додавання, ділення, остача, інкремент тощо для змінних a=17 та b=5), логічних і порівняння (>, ==, &&, ||, !), а також побітових (&, |, ^, ~, <<, >>) для змінних d та q типу unsigned char із виведенням результатів у 8-бітному двійковому форматі через %08b

```
1  #include <stdio.h>
2  #include <windows.h>
3
4  int main() {
5      SetConsoleOutputCP(CP_UTF8);
6      SetConsoleCP(CP_UTF8);
7      int a = 17, b = 5;
8      printf("Арифметичні оператори \n");
9      printf("a = %d, b = %d\n", a, b);
10     printf("a + b = %d\n", a + b);
11     printf("a - b = %d\n", a - b);
12     printf("a * b = %d\n", a * b);
13     printf("a / b = %d\n", a / b);
14     printf("a %% b = %d\n", a % b);
15     printf("++a = %d\n", ++a);
16     printf("--b = %d\n", --b);
17
18     a = 17; b = 5;
19
20     printf("Логічні оператори та порівняння\n");
21     printf("a > b: %d\n", a > b);           // Більше
22     printf("a == b: %d\n", a == b);        // Дорівнює
23     printf("(a > 10) && (b < 10): %d\n", (a > 10) && (b < 10)); // Логічне І
24     printf("(a < 5) || (b == 5): %d\n", (a < 5) || (b == 5)); // Логічне АБО
25     printf("!(a == b): %d\n", !(a == b));  // Логічне НЕ
26
27     printf("Побітові оператори\n");
28
29     unsigned char d = 15; // 00001111
30     unsigned char q = 2;  // 00000010
31
32     printf("d = %08b\n", d);
33     printf("q = %08b\n", q);
34     printf("d & q = %08b\n", d & q);        // Побітове І
35     printf("d | q = %08b\n", d | q);        // Побітове АБО
36     printf("d ^ q = %08b\n", d ^ q);        // Побітове XOR
37     printf("~d = %08b\n", (unsigned char)~d); // Побітове НЕ
38     printf("d << 2 = %08b\n", d << 2);      // Зсув вліво на 2
39     printf("d >> 1 = %08b\n", d >> 1);      // Зсув вправо на 1
40
41     return 0;
42 }
```

Результат

```
Арифметичні оператори
a = 17, b = 5
a + b = 22
a - b = 12
a * b = 85
a / b = 3
a % b = 2
++a = 18
--b = 4

Логічні оператори та порівняння
a > b: 1
a == b: 0
(a > 10) && (b < 10): 1
(a < 5) || (b == 5): 1
!(a == b): 1

Побітові оператори
d      = 00001111
q      = 00000010
d & q  = 00000010
d | q  = 00001111
d ^ q  = 00001101
~d     = 11110001011100100000000000000000
d << 2 = 00111100
d >> 1 = 00000111
```

На скріншоті зображено результат роботи програми в консолі: обчислення арифметичних операцій для $a=17$ і $b=5$, результати логічних перевірок (1 та 0), а також двійкові значення й результати побітових операцій для змінних d та q

Завдання 2

Вказівники

```
1 #include <stdio.h>
2 #include <windows.h>
3
4 int main() {
5     SetConsoleOutputCP(CP_UTF8);
6     SetConsoleCP(CP_UTF8);
7     int num;
8
9     printf("Введіть ціле число: ");
10    scanf("%d", &num);
11
12    int *ptr = &num;
13
14    printf("Значення змінної num: %d\n", num);
15    printf("Адреса змінної в пам'яті (&num): %p\n", &num);
16    printf("Адреса, яку зберігає вказівник (ptr): %p\n", ptr);
17    printf("Значення за адресою через вказівник (*ptr): %d\n", *ptr);
18
19    return 0;
20 }
```

На скріншоті зображено який демонструє роботу з вказівниками та адресами пам'яті. Програма зчитує ціле число `num` через `scanf()`, створює вказівник `ptr`, що вказує на це число, і виводить значення змінної, її адресу в пам'яті, збережену у вказівнику адресу та значення через розіменування `*ptr`

Результат

```
Введіть довільне ціле число: 4
Значення змінної num: 4
Адреса змінної в пам'яті (&num): 0000006da7fff964
Адреса, яку зберігає вказівник (ptr): 0000006da7fff964
Значення за адресою через вказівник (*ptr): 4
```

На скріншоті зображено результат виконання програми в консолі: користувач ввів число 4, після чого програма вивела значення змінної `num`, її шістнадцяткову адресу в пам'яті (`&num`), таку ж адресу, збережену у вказівнику `ptr`, та значення 4, отримане через розіменування вказівника (`*ptr`)

Завдання 3

Квадратне рівняння

```
1 #include <stdio.h>
2 #include <math.h>
3 #include <windows.h>
4
5 int main() {
6     SetConsoleOutputCP(CP_UTF8);
7     SetConsoleCP(CP_UTF8);
8     int a, b, c;
9     double D, x1, x2;
10
11     printf("Введіть коефіцієнти a, b, c (цілі числа): ");
12     scanf("%d %d %d", &a, &b, &c);
13
14
15     D = (double)(b * b - 4 * a * c);
16     printf("Дискримінант D = %.2lf\n", D);
17
18     if (D > 0) {
19
20         x1 = (-b + sqrt(D)) / (2.0 * a);
21         x2 = (-b - sqrt(D)) / (2.0 * a);
22         printf("Рівняння має 2 корені:\n");
23         printf("x1 = %.2lf\n", x1);
24         printf("x2 = %.2lf\n", x2);
25     }
26     else if (D == 0) {
27         x1 = (double)(-b) / (2.0 * a);
28         printf("Рівняння має 1 корінь:\n");
29         printf("x = %.2lf\n", x1);
30     }
31     else {
32         printf("Дійсних коренів немає (D < 0)\n");
33     }
34
35     return 0;
36 }
```

На скріншоті зображено код для розв'язання квадратного рівняння. Програма зчитує коефіцієнти a , b , c , обчислює дискримінант D та за допомогою умовних операторів `if-else` знаходить корені рівняння: два корені при $D > 0$, один корінь при $D = 0$ або виводить повідомлення про відсутність дійсних коренів при $D < 0$.

Результат

```
Введіть коефіцієнти a, b, c (цілі числа): 2 4 6
Дискримінант D = -32.00
Дійсних коренів немає (D < 0)
```

На скріншоті зображено результат виконання програми в консолі: користувач ввів коефіцієнти 2, 4, 6, після чого програма обчислила від'ємний дискримінант ($D = -32.00$) та вивела повідомлення про відсутність дійсних коренів ($D < 0$).

Висновок

У ході виконання лабораторної роботи було опановано роботу з базовими типом даних, специфікаторами формату `printf()` та розширеним введенням рядків із пробілами через `scanf("%[^\n]")`. На практиці досліджено використання арифметичних, логічних і побітових операторів мови Cі, а також виведення чисел у різних системах числення (десятковій, двійковій, вісімковій та шістнадцятковій).

Окрім цього, засвоєно принципи адресації та роботи з вказівниками (отримання адреси змінної `&` та її розіменування `*`), а також реалізовано алгоритм розв'язання квадратного рівняння з аналізом дискримінанта та виведенням результатів у вигляді структурованих консольних даних і таблиць.